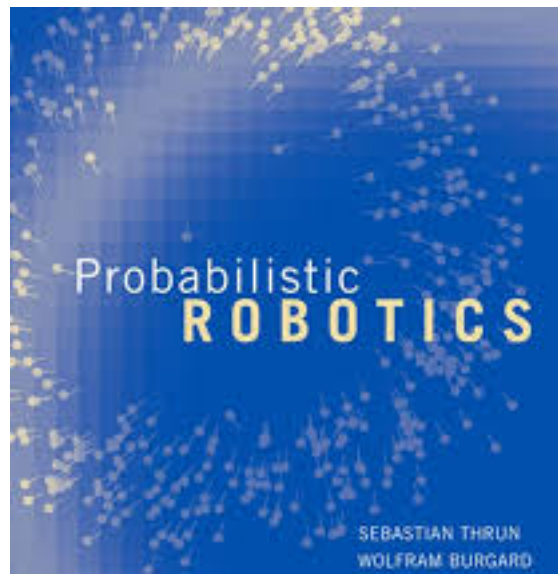


Robótica Probabilística

Estimação de Estado Recursiva

Estimação de Estado Recursiva

Capítulo 2 do livro *Probabilistic Robotics*, de Sebastian Thrun, Wolfram Burgard e Dieter Fox



Estimação de Estado: Núcleo da Robótica Probabilística

- ▶ Ideia central da robótica probabilística: estimar o estado a partir de dados de sensores
- ▶ **Estimação de estado:** aborda o problema de estimar grandezas não diretamente observáveis, mas que podem ser inferidas a partir de dados de sensores
- ▶ Em muitas aplicações robóticas, decidir o que fazer é relativamente simples se certas grandezas forem conhecidas
 - ▶ Ex: mover um robô móvel é relativamente fácil se a localização exata do robô e dos obstáculos próximos forem conhecidas
- ▶ **Problema:** essas variáveis não são diretamente mensuráveis
 - ▶ O robô precisa confiar em seus sensores para coletar essa informação
 - ▶ Sensores carregam apenas informação parcial sobre essas grandezas, e suas medições são corrompidas por ruído
 - ▶ Estimação de estado busca recuperar as variáveis de estado a partir dos dados
 - ▶ Algoritmos probabilísticos de estimação de estado calculam **distribuições de crença** (*belief distributions*) sobre os possíveis estados do mundo

Interação Entre Robô e Ambiente

Interação Robô-Ambiente

- ▶ Ambiente (ou mundo) de um robô: sistema dinâmico que possui estado interno
- ▶ O robô adquire informação sobre o ambiente por meio de seus **sensores**
 - ▶ Limitações: são ruidosos; muitas grandezas não podem ser sensoriadas diretamente
 - ▶ Consequência: o robô mantém uma **crença interna** (*belief*) sobre o estado do ambiente
- ▶ O robô também pode influenciar seu ambiente por meio de **atuadores**
 - ▶ O efeito de agir sobre o ambiente é frequentemente algo imprevisível

Estado (State)

- ▶ Ambientes são caracterizados por **estado**: conjunto de todos os aspectos do robô e de seu ambiente que podem impactar o futuro

Tipos de Estado

- ▶ Estado dinâmico: muda ao longo do tempo (ex: localização de pessoas)
- ▶ Estado estático: não muda (ex: localização de paredes na maioria dos edifícios)
- ▶ Também incluídas: pose, velocidade, se os sensores estão funcionando corretamente, entre outras

Notação

- ▶ Estado é definido pelo vetor x ; as variáveis específicas incluídas em x dependem do contexto
- ▶ Estado no instante t : denotado x_t
- ▶ Exemplo: $x = \langle 0,58, 0,78, 0,87, \dots \rangle$

Variáveis de Estado Típicas em Robótica

1. **Pose do Robô** (*kinematic state*) — localização e orientação em relação a um referencial global
 - ▶ Robôs móveis: 6 variáveis (3 coordenadas + 3 orientações angulares — *pitch*, *roll*, *yaw*)
 - ▶ Robôs em ambientes planos: geralmente 3 variáveis (2 de localização + *heading/yaw*)
2. **Configuração dos Atuadores** — ex: articulações de manipuladores robóticos
3. **Velocidade do Robô e de suas Articulações** (*dynamic state*) — até 6 variáveis de velocidade
4. **Localização e Características de Objetos no Ambiente** — árvores, paredes; podem assumir a forma de marcos (*landmarks*); o número de variáveis varia de poucas dezenas a centenas de bilhões, conforme a granularidade do modelo
5. **Localização e Velocidades de Objetos e Pessoas em Movimento** — o robô raramente é o único ator em movimento no ambiente
6. **Outras Variáveis** — ex: sensor quebrado, nível de bateria

Completude do Estado (State Completeness)

- ▶ Um estado x_t é **completo** se for o melhor preditor do futuro
- ▶ Completude significa: conhecimento de estados, medições ou controles passados não carrega informação adicional que ajude a prever o futuro com mais precisão que o estado atual
- ▶ Completude **não** exige que o futuro seja função determinística do estado
 - ▶ O futuro pode ser estocástico, mas nenhuma variável anterior a x_t pode influenciar a evolução estocástica dos estados futuros, a menos que essa dependência seja mediada pelo próprio x_t
- ▶ Processos temporais que atendem a essas condições são conhecidos como **Cadeias de Markov** (*Markov chains*)

Importância Prática: Estado Incompleto

- ▶ A noção de completude de estado é principalmente de importância teórica
- ▶ Na prática, é impossível especificar um estado completo para qualquer sistema robótico realista
 - ▶ Incluiria não só o ambiente, mas também o próprio robô, o conteúdo de sua memória computacional, “estados mentais” das pessoas ao redor etc. — difíceis (ou impossíveis) de obter
- ▶ Implementações práticas selecionam um pequeno subconjunto das variáveis de estado
- ▶ Tal estado é chamado de **estado incompleto**

Tempo

- ▶ Na maioria dos problemas interessantes de robótica, o estado muda ao longo do tempo
- ▶ Em robótica probabilística, o tempo é tratado como **discreto** — eventos ocorrem em passos: $0, 1, 2, \dots$
- ▶ Se o robô inicia sua operação em um ponto distinto no tempo, esse instante será denotado $t = 0$

Interação Robô-Ambiente

Existem dois tipos fundamentais de interação entre robô e ambiente:

- ▶ Influenciar o estado do ambiente por meio de **atuadores**
- ▶ Coletar informação sobre o estado por meio de **sensores**

Ambos podem ocorrer simultaneamente, mas por razões didáticas serão distinguidos nesta disciplina.

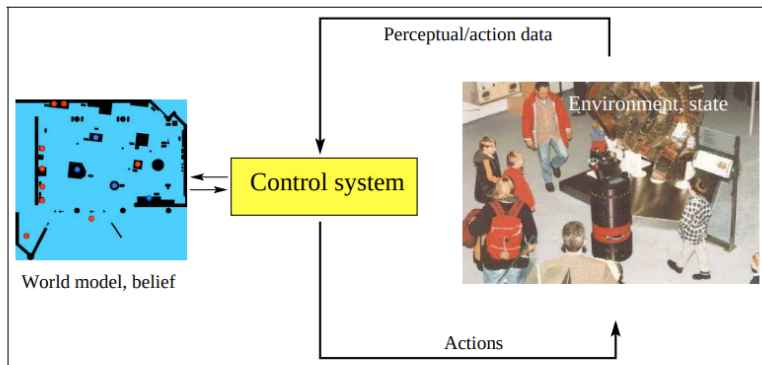


Figure 2.1 Robot Environment Interaction.

Dados: Medições e Controle

- ▶ O robô pode manter registro de todas as medições e ações de controle passadas — coletivamente chamado de **dados** (*data*)
- ▶ Existem duas correntes de dados diferentes, correspondendo aos dois tipos de interação:
 - ▶ Dados de Medição (*Measurement Data*)
 - ▶ Dados de Controle (*Control Data*)

Dados de Medição (Measurement Data)

- ▶ Fornecem informação sobre o estado momentâneo do ambiente
 - ▶ Exemplos: imagens de câmera, varreduras de alcance
 - ▶ Efeitos de timing pequenos geralmente ignorados (ex: lidar escaneia sequencialmente, mas assume-se instante único)
- ▶ Assume-se exatamente uma medição por vez

Notação: z_t (medição no instante t)

Sequência de medições:

$$z_{t_1:t_2} = z_{t_1}, z_{t_1+1}, \dots, z_{t_2} \quad (t_1 \leq t_2)$$

Dados de Controle (Control Data)

- ▶ Carregam informação sobre a mudança de estado no ambiente
- ▶ Exemplo típico: velocidade do robô (ex: 10 cm/s por 5s → pose avança ~50 cm)
- ▶ Fonte alternativa: odômetros (sensores que medem a revolução das rodas) — tratados como dado de controle, pois a informação principal refere-se à mudança de estado

Notação: u_t (mudança de estado no intervalo $(t-1, t]$)

Sequência de controles:

$$u_{t_1:t_2} = u_{t_1}, u_{t_1+1}, \dots, u_{t_2} \quad (t_1 \leq t_2)$$

A passagem do tempo, mesmo sem ação, constitui tecnicamente informação de controle. Assume-se um item de controle por passo de tempo.

Distinção entre Medição e Controle

- ▶ Os dois tipos de dados desempenham papéis fundamentalmente diferentes
- ▶ **Percepção**: fornece informação sobre o estado → tende a **aumentar** o conhecimento do robô
- ▶ **Movimento**: tende a induzir **perda** de conhecimento, devido ao ruído da atuação e à estocasticidade do ambiente (às vezes um controle pode tornar o robô mais certo)
- ▶ Essa distinção não implica que ações e percepções ocorrem em momentos separados — na prática ocorrem concorrentemente (muitos sensores afetam o próprio ambiente); a separação é feita por conveniência didática

Leis Generativas Probabilísticas

As Duas Probabilidades Fundamentais

Probabilidade de transição de estado

$$p(x_t \mid x_{t-1}, u_t)$$

- ▶ Especifica como o estado ambiental evolui em função dos controles do robô
- ▶ Ambientes robóticos são estocásticos: é uma distribuição de probabilidade, não uma função determinística

Probabilidade de medição (*measurement probability*)

$$p(z_t \mid x_t)$$

- ▶ Especifica a lei segundo a qual as medições z são geradas a partir do estado x
- ▶ Medições geralmente são projeções ruidosas do estado

Modelo Generativo Temporal

- ▶ Probabilidade de transição + probabilidade de medição descrevem juntas o sistema dinâmico estocástico do robô e seu ambiente
- ▶ Estado em t depende estocasticamente do estado em $t - 1$ e do controle u_t
- ▶ Medição z_t depende estocasticamente do estado em t
- ▶ Esse modelo é conhecido como **Modelo Oculto de Markov** (*Hidden Markov Model* — HMM) ou **Rede Bayesiana Dinâmica** (*Dynamic Bayes Network* — DBN)
- ▶ Para especificar o modelo completamente, também é necessária a distribuição de estado inicial $p(x_0)$

Distribuições de Crença

Distribuições de Crença (Belief Distributions)

- ▶ Conceito-chave em robótica probabilística: **crença** (*belief*)
- ▶ Reflete o conhecimento interno do robô sobre o estado do ambiente
- ▶ O estado não pode ser medido diretamente
 - ▶ Ex: pose do robô $x = \langle 14, 12, 12, 7, 0, 755 \rangle$ — mas o robô geralmente não pode conhecer sua pose exata (nem mesmo com GPS!)
 - ▶ O robô precisa **inferir** sua pose a partir de dados
- ▶ Distinção importante: **estado verdadeiro** vs. **crença interna**

Representação Matemática

- ▶ A robótica probabilística representa crenças por meio de distribuições de probabilidade condicionais
- ▶ Distribuição de crença: atribui uma probabilidade (ou densidade) a cada hipótese possível sobre o estado verdadeiro

Notação

Crença sobre a variável de estado x_t : $bel(x_t)$, abreviação de:

$$bel(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$$

Distribuição de probabilidade sobre o estado x_t no instante t , condicionada a todas as medições passadas ($z_{1:t}$) e controles passados ($u_{1:t}$). Calculada **após** incorporar a medição z_t .

Predição (Prediction)

Às vezes é útil calcular uma posterior antes de incorporar z_t , logo após executar o controle u_t :

$$\overline{bel}(x_t) = p(x_t \mid z_{1:t-1}, u_{1:t})$$

- ▶ Chamada de **predição**, no contexto de filtragem probabilística
- ▶ Reflete que $\overline{bel}(x_t)$ prediz o estado em t com base na posterior do estado anterior, antes de incorporar a medição
- ▶ Calcular $bel(x_t)$ a partir de $\overline{bel}(x_t)$ é chamado de **correção** ou **atualização de medição** (*measurement update*)

Filtro de Bayes

Algoritmo do Filtro de Bayes

- ▶ Algoritmo mais geral para calcular crenças: **Filtro de Bayes** (*Bayes filter*)
- ▶ Calcula a distribuição de crença *bel* a partir de dados de medição e controle
- ▶ **Recursivo**: $bel(x_t)$ é calculada a partir de $bel(x_{t-1})$
- ▶ **Entrada**: crença *bel* em $t - 1$, controle mais recente u_t , medição mais recente z_t
- ▶ **Saída**: crença $bel(x_t)$ no instante t

Algorithm Filtro de Bayes ($bel(x_{t-1}), u_t, z_t$)

```
1: for all  $x_t$  do  
2:    $\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1}) bel(x_{t-1}) dx$   
3:    $bel(x_t) = \eta p(z_t \mid x_t) \overline{bel}(x_t)$   
4: end for  
5: return  $bel(x_t)$ 
```

Passos Essenciais

1. Processamento do Controle (*Control Update* / Predição)

- ▶ Calcula uma crença sobre x_t com base na crença prévia sobre x_{t-1} e no controle u_t
- ▶ $\overline{bel}(x_t)$: integral (ou soma) do produto de duas distribuições — a crença anterior em x_{t-1} , e a probabilidade de que u_t induza a transição de x_{t-1} para x_t

2. Atualização de Medição (*Measurement Update*)

- ▶ Multiplica $\overline{bel}(x_t)$ pela probabilidade de que z_t tenha sido observada, para cada estado hipotético x_t
- ▶ O produto geralmente não integra a 1 — normalizado pela constante η
- ▶ Resultado final: $bel(x_t)$, retornado ao final do algoritmo

Crença Inicial

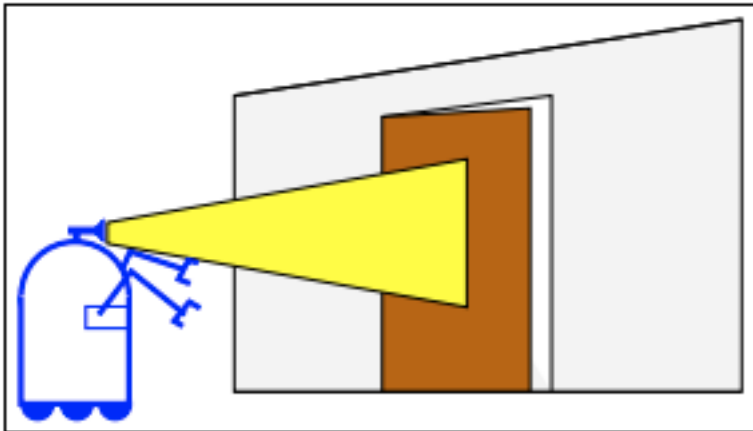
- ▶ Para calcular a posterior recursivamente, o algoritmo exige uma crença inicial $bel(x_0)$ em $t = 0$
- ▶ Se x_0 é conhecido com certeza: distribuição de massa pontual (probabilidade 1 no valor correto, 0 no resto)
- ▶ Se x_0 é totalmente desconhecido: distribuição uniforme sobre o domínio de x_0
- ▶ Conhecimento parcial de x_0 : distribuições não uniformes
- ▶ Na prática, os casos mais comuns são: conhecimento total ou ignorância total

Limitações Práticas

- ▶ O algoritmo, na forma apresentada, só pode ser implementado diretamente para problemas de estimação muito simples
- ▶ Requisitos:
 - ▶ Capacidade de realizar a integração (predição) e a multiplicação (atualização) em forma fechada, **ou**
 - ▶ Restringir-se a espaços de estado finitos, tornando a integral uma soma finita

Exemplo

Um robô móvel estimando o estado de uma porta



Cenário

- ▶ Robô estima o estado de uma porta usando câmera
- ▶ Estados possíveis: **aberta** ou **fechada**
- ▶ Apenas o robô pode mudar o estado da porta
- ▶ Crença inicial (desconhecimento total):

$$bel(X_0 = aberta) = 0,5 \quad bel(X_0 = fechada) = 0,5$$

Modelo de Medição (Sensor Ruidoso)

$$p(\text{percebe aberta} \mid \text{está aberta}) = 0,6$$

$$p(\text{percebe fechada} \mid \text{está aberta}) = 0,4$$

$$p(\text{percebe aberta} \mid \text{está fechada}) = 0,2$$

$$p(\text{percebe fechada} \mid \text{está fechada}) = 0,8$$

Sensor é relativamente confiável para detectar porta fechada (erro de 0,2); menos confiável para porta aberta (erro de 0,4).

Modelo de Transição de Estado (Ação de Controle)

- ▶ Ação “**empurrar**” (*push*): se já aberta, permanece aberta (prob. 1,0); se fechada, 0,8 de chance de abrir
- ▶ Ação “**não fazer nada**”: estado do mundo não muda (probabilidade 1,0 de permanecer no mesmo estado), esteja aberta ou fechada

Passo 1

$t = 1$, controle = não fazer nada, medição = percebe aberta

Predição (Control Update)

$$\overline{bel}(X_1 = \text{aberta}) = 1,0 \cdot 0,5 + 0 \cdot 0,5 = 0,5 \quad \overline{bel}(X_1 = \text{fechada}) = 0 \cdot 0,5$$

(a crença não muda, pois “não fazer nada” não altera o estado do mundo)

Correção (Measurement Update)

$$bel(X_1 = \text{aberta}) = \eta \cdot 0,6 \cdot 0,5 = \eta \cdot 0,3 \quad bel(X_1 = \text{fechada}) = \eta \cdot 0,2 \cdot 0,$$

Normalizador: $\eta = (0,3 + 0,1)^{-1} = 2,5$. Resultado:

$$bel(X_1 = \text{aberta}) = 0,75, \quad bel(X_1 = \text{fechada}) = 0,25$$

Passo 2

$t = 2$, controle = empurrar, medição = percebe aberta

Predição

$$\overline{bel}(X_2 = \text{aberta}) = 1 \cdot 0,75 + 0,8 \cdot 0,25 = 0,95 \quad \overline{bel}(X_2 = \text{fechada}) = 0$$

Correção

$$bel(X_2 = \text{aberta}) = \eta \cdot 0,6 \cdot 0,95 \approx 0,983 \quad bel(X_2 = \text{fechada}) = \eta \cdot 0,2 \cdot 0$$

Interpretação Final

- ▶ Após duas iterações, o robô acredita com **98,3%** de probabilidade que a porta está aberta
- ▶ Apesar de parecer alta o suficiente para aceitar como certa, essa abordagem pode gerar custos desnecessariamente altos
- ▶ Se confundir porta fechada com aberta tem custo alto (ex: colisão), é essencial considerar ambas as hipóteses no processo de decisão, por mais improvável que uma pareça
- ▶ Analogia: imagine pilotar um avião no piloto automático com 98,3% de chance percebida de não colidir!

Representação e Computação

Representação e Computação

- ▶ Filtros de Bayes são implementados de diversas formas na robótica probabilística
- ▶ Existe grande variedade de técnicas e algoritmos, todos derivados do filtro de Bayes
- ▶ Cada técnica baseia-se em diferentes suposições sobre medição, transição de estado e crença inicial
- ▶ Essas suposições geram diferentes tipos de distribuições posteriores, com algoritmos de características computacionais distintas

Necessidade de Aproximação

- ▶ Regra geral: técnicas exatas para calcular crenças existem apenas para casos altamente especializados
- ▶ Em problemas robóticos gerais, crenças precisam ser **aproximadas**
- ▶ A natureza da aproximação tem grandes implicações na complexidade do algoritmo
- ▶ Encontrar aproximação adequada é geralmente desafiador — não há resposta única ideal para todos os problemas

Critérios de Escolha da Aproximação (Trade-offs)

1. Eficiência Computacional

- ▶ Aproximações Gaussianas lineares: tempo polinomial em relação à dimensão do espaço de estado; outras podem exigir tempo exponencial
- ▶ Técnicas baseadas em partículas: característica *any-time*, trocando precisão por eficiência

2. Precisão da Aproximação

- ▶ Gaussianas lineares: limitadas a distribuições unimodais
- ▶ Histogramas: aproximam distribuições multimodais, com precisão limitada
- ▶ Partículas: ampla variedade de distribuições, mas exigem muitas partículas para atingir a precisão desejada

Critérios de Escolha da Aproximação (cont.)

3. Facilidade de Implementação

- ▶ Depende da forma de $p(z_t \mid x_t)$ e $p(x_t \mid u_t, x_{t-1})$
- ▶ Representações por partículas: implementações surpreendentemente simples para sistemas não lineares complexos — uma das razões de sua popularidade

Nas próximas aulas veremos algoritmos concretos e implementáveis, com desempenhos distintos em relação a esses critérios.